Mathematical Modelling Project

Adaptive Debt Collection Routing under Payment Uncertainty

Nguyen Thang Phuc
Student ID: 20252263M
Phuc.NT252263M@sis.hust.edu.vn

Dinh Thi Lan Huong
Student ID: 20261261M
Huong.DTL261261M@sis.hust.edu.vn

Supervisor: Viet Hung NGUYEN | viet_hung.nguyen@uca.fr

1 Introduction

Debt collection requires financial institutions to allocate limited staff time among customers located in different geographical areas. When in-person visits are necessary, collection officers must decide which customers to visit, in what order, and on which days. These decisions influence both travel requirements and the amount of money that may be recovered. Visiting every customer is often impractical because travel and service times must fit within daily working-hour limits. Furthermore, a visit does not guarantee payment, and an unsuccessful visit may require a subsequent attempt.

This project considers a multi-officer, multi-day debt collection routing problem in Hanoi. A team of collection officers operates from a common branch over a finite planning horizon. Each customer is associated with a target collection amount, an assumed payment probability, a service duration, and a geographical location. Each officer's daily route starts and ends at the branch, and total travel and service time must remain within the available working hours. Customers may be revisited subject to a maximum visit allowance, but a successfully collected amount must not be counted more than once.

The motivation is to allocate collection effort effectively under limited operational capacity and uncertain payment outcomes. Prioritizing nearby customers alone may overlook larger expected recoveries, whereas prioritizing large debts alone may lead to excessive travel or repeated unsuccessful visits. The problem therefore combines customer selection, visit scheduling, and route construction. The primary objective is to maximize expected recovered money. A secondary objective is to minimize total travel distance while preserving the primary objective value within a specified numerical tolerance.

The planning problem is formulated as a mixed-integer linear programming model and implemented in Julia using JuMP and the HiGHS solver. Heuristic approaches, including a genetic algorithm, tabu search, and adaptive large neighborhood search, are also investigated. The experimental study is designed to compare solution quality, computational time, and optimality gaps when valid solver bounds are available.

Two planning policies are considered. A static policy constructs a plan for the entire horizon before execution. A rolling-horizon policy updates customer states after each day and reoptimizes the remaining plan. Customers who have paid are removed from future visits, while unsuccessful visits reduce their remaining visit allowances. Simulated payment outcomes provide a common basis for comparing these policies.

The study uses a synthetic dataset informed by public demographic and credit-related data, together with geographical and road-network information. Customer accounts and payment outcomes are simulated and do not represent an actual bank's collection records. Payment probabilities are modelling assumptions rather than estimates validated against historical collection outcomes. The project therefore evaluates the behaviour and trade-offs of the proposed methods under controlled scenarios, rather than claiming verified improvements in real banking operations.

2 Mathematical Model

2.1 Problem Assumptions

A successful visit to customer i collects the full target amount w_i and an unsuccessful one collects nothing. p_{it} is the probability that customer i pays when visited on day t, given no earlier payment; outcomes of different visits are conditionally independent and p_{it} decreases with t. Customer i is visited at most m_i times over the horizon ($m_i = 2$ in all experiments) and at most once per day, by one officer; different visits may be made by different officers. All officers can visit all customers and have equal daily budgets. Every active officer leaves the branch once per day and returns before the budget, which covers travel and service time, is used up. A customer who has paid receives no further visits: in the planning model this is captured by the expected value of a visit schedule, and during execution by skipping the customer (Section 2.6).

2.2 Input Data and Notation

Table 1 lists the notation. Money is expressed in million VND, time in hours and distance in kilometers. Travel times and distances come from shortest (fastest) paths on a directed motorbike road network, so in general $d_{ij} \neq d_{ji}$ and $\tau_{ij} \neq \tau_{ji}$.

Table 1: Main notation.

Symbol	Description
N	Set of customers
K	Set of collection officers
T	Set of planning days
$V = N \cup \{0\}$	Routing nodes, where 0 is the branch
$A = \{(i,j) \in V \times V : i \neq j\}$	Arcs of the complete directed routing graph
$n = \lvert N \rvert$	Number of customers
w_i	Target collection amount of customer i (million VND)
p_{it}	Payment probability for customer i on day t, given no earlier payment
d_{ij}	Travel distance from node i to node j (km)
τ_{ij}	Travel time from node i to node j (hours)
s_i	Service duration at customer i (hours)
B_{kt}	Working-time budget of officer k on day t (hours; 8 h in all instances)
m_i	Maximum number of visits to customer i
$\mathcal{S}_i$	Feasible visit schedules of customer i: $\{S \subseteq T : 1 \leq \lvert S \rvert \leq m_i\}$
a_{tS}	1 if $t \in S$, 0 otherwise
v_{iS}	Precomputed expected collection of customer i under schedule S (million VND)

2.3 Decision Variables

All variables are indexed by $i, j \in V$, $k \in K$, $t \in T$ and $S \in \mathcal{S}_i$.

- $x_{ijkt} \in \{0,1\}$, $(i,j) \in A$: equals 1 if officer k travels directly from node i to node j on day t (arc variables).
- $y_{ikt} \in \{0,1\}$, $i \in N$: equals 1 if officer k visits customer i on day t (visit assignment).
- $z_{kt} \in \{0,1\}$: equals 1 if officer k performs a route on day t (route activation).
- $\lambda_{iS} \in \{0,1\}$, $i \in N$: equals 1 if customer i is visited exactly on the set of days S (schedule selection).
- $u_{ikt} \geq 0$, $i \in N$: position of customer i in the route of officer k on day t; used only for subtour elimination (Miller–Tucker–Zemlin variables [5]).

2.4 Objective Function

With full-or-zero payment and conditionally independent visits, the probability that customer i pays at least once under schedule S is $1 - \prod_{t \in S}(1 - p_{it})$, so its expected collection is

$$v_{iS} = w_i \Big[1 - \prod_{t \in S} (1 - p_{it}) \Big]. \tag{1}$$

This value is not additive over the visits: a second visit only adds value if the first one fails. Expressing it directly in terms of y_{ikt} would require products of binary variables. Instead, v_{iS} is computed before solving for every schedule in $\mathcal{S}_i$, and the model selects one schedule per customer. The objective is therefore linear:

$$\max \; Z = \sum_{i \in N} \sum_{S \in \mathcal{S}_i} v_{iS} \, \lambda_{iS}. \tag{2}$$

With $H = 5$ days and $m_i = 2$, each customer has $|\mathcal{S}_i| = \binom{5}{1} + \binom{5}{2} = 15$ schedules.

Secondary objective. Let $Z^\star$ be the value obtained in the first solve. A second solve keeps the expected collection within a relative tolerance $\varepsilon = 10^{-6}$ and minimizes the total distance:

$$\sum_{i \in N} \sum_{S \in \mathcal{S}_i} v_{iS} \lambda_{iS} \;\geq\; Z^\star - \varepsilon \max(1, |Z^\star|), \tag{3}$$

$$\min \sum_{t \in T} \sum_{k \in K} \sum_{(i,j) \in A} d_{ij} \, x_{ijkt}. \tag{4}$$

The second solve starts from the first solution and has its own time limit. In the experiments it is run only when the first solve is proved optimal, with the same time limit as the first solve. In the heuristics, distance is always used as a lexicographic tie-breaker between solutions with equal expected value.

2.5 Constraints

$$\sum_{S \in \mathcal{S}_i} \lambda_{iS} \leq 1 \qquad \forall i \in N, \tag{5}$$

$$\sum_{k \in K} y_{ikt} = \sum_{S \in \mathcal{S}_i} a_{tS} \lambda_{iS} \qquad \forall i \in N, \; t \in T, \tag{6}$$

$$\sum_{j \in N} x_{0jkt} = z_{kt}, \qquad \sum_{i \in N} x_{i0kt} = z_{kt} \qquad \forall k \in K, \; t \in T, \tag{7}$$

$$\sum_{j \in V \setminus \{i\}} x_{ijkt} = y_{ikt}, \qquad \sum_{j \in V \setminus \{i\}} x_{jikt} = y_{ikt} \qquad \forall i \in N, \; k \in K, \; t \in T, \tag{8}$$

$$\sum_{(i,j) \in A} \tau_{ij} x_{ijkt} + \sum_{i \in N} s_i y_{ikt} \leq B_{kt} z_{kt} \qquad \forall k \in K, \; t \in T, \tag{9}$$

$$u_{ikt} - u_{jkt} + n \, x_{ijkt} \leq n - 1 + n(1 - y_{jkt}) \qquad \forall i, j \in N, \; i \neq j, \; k \in K, \; t \in T, \tag{10}$$

$$y_{ikt} \leq u_{ikt} \leq n \, y_{ikt} \qquad \forall i \in N, \; k \in K, \; t \in T, \tag{11}$$

$$x_{ijkt}, \, y_{ikt}, \, z_{kt}, \, \lambda_{iS} \in \{0, 1\}, \quad u_{ikt} \geq 0. \tag{12}$$

Constraint (5) gives each customer at most one schedule (a customer may be skipped; m_i is enforced through $\mathcal{S}_i$). Constraint (6) makes exactly one officer visit customer i on each day of its schedule and nobody on other days. Constraints (7)–(8) make an active officer leave and re-enter the branch once and give every visited customer one incoming and one outgoing arc on the route of its officer. Constraint (9) limits travel plus service time to B_{kt}. In the MTZ

constraints (10)–(11), $x_{ijkt} = 1$ with j visited forces $u_{jkt} \geq u_{ikt} + 1$, so no cycle can avoid the branch; the term $n(1 - y_{jkt})$ relaxes the constraint when j is not on the route.

Symmetry breaking. Officers with equal budgets are interchangeable, so the search tree contains relabelled copies of every solution. For consecutive officers with $B_{kt} = B_{k+1,t}$ the model adds

$$z_{kt} \geq z_{k+1,t} \qquad k = 1, \ldots, |K| - 1,\ t \in T, \tag{13}$$

which does not change the optimal value and is used in all experiments.

2.6 Rolling-Horizon Operation

The model above produces a *static* plan for the whole horizon. During execution, payment outcomes are revealed day by day. The state known at the start of day t is (P_t, a_t), where P_t is the set of customers who have already paid and a_{it} is the number of visits made to customer i before day t. The eligible customers are $A_t = \{i \in N : i \notin P_t,\ a_{it} < m_i\}$.

Each morning $t = 1, \ldots, H$, the rolling-horizon policy builds the remaining problem (customers A_t, days $t, \ldots, H$, remaining visit allowances $m_i - a_{it}$), solves model (2)–(12) on it, executes only the routes of day t and records the simulated payments. The probabilities are not re-estimated: p_{it} is already conditional on no earlier payment, so it stays valid for every customer in A_t. Two reference policies are compared with it. The *static* policy executes the initial plan and skips customers who have already paid, without inserting new ones. The *myopic* policy re-plans each morning but optimizes day t only ($H = 1$). All three policies produce feasible routes. The rolling horizon is not a fully adaptive stochastic model: each re-solve assumes that the remaining plan will be executed without further re-planning, so it is not an optimal policy of the multi-stage problem.

3 Solution Methods

3.1 MILP Solution with HiGHS

The model is built with JuMP [7] and solved by the branch-and-cut algorithm of HiGHS [8] (presolve, LP relaxation, cutting planes, primal heuristics and branching). The solver stops at a relative gap of 10^{-4} or at the time limit, and its random seed is the seed of the run. The status, bound and node count are read after the main solve.

Warm start. In the warm-started variant, the ALNS solution of the same seed is passed as a MIP start (values of x, y, z, u and the matching λ_{iS}). If HiGHS stops without a solution at least as good as the start, the ALNS solution is returned and labelled as such; this happened in 6 of the 70 five-day runs (1 on INS_S with $n = 50$, 2 on REP1_S, 3 on REP2_S). The bound remains valid in all cases.

Shared heuristic framework. The three metaheuristics below use the same representation: an assignment matrix $G \in \{0, 1, \ldots, |K|\}^{n \times H}$, where $g_{it} = 0$ means that customer i is not visited on day t and $g_{it} = k$ means that officer k visits them. A cell holds one value, so a customer cannot be visited by two officers on the same day. For every officer–day pair, the route through the assigned customers is built by nearest-neighbor from the branch and improved by 2-opt, where every reversal is evaluated on the full (asymmetric) distance. A shared *repair* procedure restores feasibility: if a customer has more than m_i visits, only the m_i days with the highest p_{it} are kept; if a route exceeds B_{kt}, the visit with the smallest marginal loss $v_{iS_i} - v_{i,S_i \setminus \{t\}}$ is removed and the route rebuilt, until the route fits. Solutions are compared lexicographically: higher expected collection first, then shorter total distance. Because the order within a route is built by a rule

rather than optimized, the heuristics may miss routes that the MILP can find, and they provide no optimality certificate.

All three metaheuristics stop after at most 300 iterations (generations) or after 50 consecutive iterations without improving the best solution.

Genetic algorithm. An individual is one matrix G. Each cell of an initial individual is a visit with probability 0.35, assigned to a random officer. Parents are chosen by tournaments of size 3, children are formed by uniform crossover and mutated cell by cell with probability 0.03, every child is repaired, and the 4 best individuals survive unchanged. The population size is 80.

Tabu search. The initial solution inserts visits (i, t) greedily in decreasing order of $w_i p_{it}/(s_i + \tau_{0i} + \tau_{i0})$. A move sets one cell (i, t) to "no visit" or to another officer, i.e. adds, removes or reassigns one visit. Each iteration samples 40 moves, repairs them and moves to the best admissible one, even if it is worse. Returning a cell to its previous value is tabu for 15 iterations, unless it gives a new best solution.

ALNS. ALNS [6] removes about 20% of the visits and rebuilds the plan. The destroy operators are random, worst-value (lowest marginal value), related (a random customer and its nearest neighbors) and route removal; the repair operators are greedy insertion (largest gain in expected value, ties broken by distance) and regret-2 insertion, both run until no feasible insertion remains. Operators are chosen by roulette wheel; they score 10, 5 or 1 for a new best, an improving or an accepted solution, and every 25 iterations the weights are updated as $w \leftarrow 0.8\, w + 0.2\, \bar{\pi}$. Acceptance is simulated annealing on $Z/\sum_i w_i - 10^{-4} \times \text{km}$ (initial temperature 0.05, cooling 0.995); the best solution is kept by the lexicographic comparison.

4 Numerical Experiments

4.1 Dataset and Test Instances

The dataset [1] combines three kinds of inputs.

Observed public data. The road network, the 2025 ward boundaries and populations, building density and the branch location (a bank point of interest in central Hanoi) come from OpenStreetMap [2]. After keeping the largest strongly connected motorbike network, the graph has 57,242 nodes and 130,220 directed edges, 6.6% of them one-way. Demographic profiles (gender, age, marital status, education, occupation) are taken unchanged from NVIDIA Nemotron-Personas-Vietnam [3].

Synthetic inputs informed by public data. Customer locations are sampled in 27 wards with quotas proportional to ward population, at road nodes weighted by nearby building density. The shape of the credit-limit and card-utilization distributions is taken from the UCI Default of Credit Card Clients dataset [4] (customers at least two months late), and the shares of debt groups 2–5 follow published Vietnamese bank reports. Loan amounts, interest rates and terms of other products are drawn from assumed log-normal distributions. The target amount w_i is the overdue amount.

Modelling assumptions. The payment probabilities $p_{it} = h\, p_0(g)\, (1 - 0.3 f_i)\, (1 - \delta(g))^{t-1}$, capped at 0.95, depend only on the debt group g, the position f_i of the customer's days past due within the group, and the day t; $h = 0.5/1/1.5$ defines the *low*, *base* and *high* scenarios (THAP, CO_SO, CAO). Road speeds by road type, service times, the 480-minute working day and the planning period (five working days) are also assumed. The probabilities are not calibrated against real collection outcomes.

The model-layer graph is complete and directed: $|V| = n + 1$ nodes and $|A| = n(n + 1)$ arcs, where each arc carries the time and distance of the fastest road path, computed with Dijkstra's

algorithm. The matrix is asymmetric for 95.8% of the node pairs. Figure 1 illustrates the two graph layers.

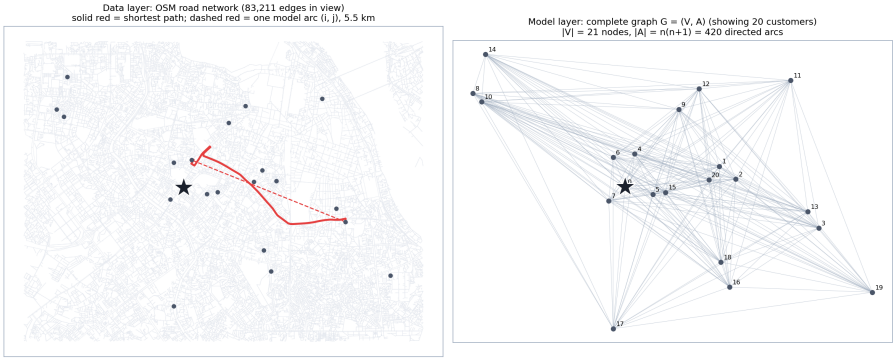


Figure 1: Left: the road network (data layer) and the fastest road path behind one model arc (i, j). Right: the complete directed graph of the model (model layer) for the first 20 customers of INS_XS.

Five customer sets were generated with different seeds: MAIN (500 customers) and REP1–REP4 (200 each), from which nested instances are taken (INS_XS, INS_S, INS_M, INS_L with 30, 80, 200 and 500 customers from MAIN; REPk_S and REPk_M with 80 and 200 from REPk). To isolate the effect of size, the experiments fix $|K| = 2$, $B_{kt} = 8$ h, $m_i = 2$ and the base scenario, and use the first n customers of INS_XS ($n = 10, 20, 30$) or INS_S ($n = 50, 80$). REP1_S–REP4_S (one day) and REP1_S–REP2_S (five days) are added at $n = 80$ to check that the conclusions do not depend on one customer set.

4.2 Computational Environment

All experiments ran on a laptop with an Intel Core i7-1165G7 (2.80 GHz, 4 cores), 8 GB of memory and Windows 10, using Julia 1.13.2, JuMP 1.32.1 and HiGHS.jl 1.26.0 (HiGHS 1.15.1) with default solver settings, under which the MIP search is essentially serial. Two experiments ran in parallel as separate single-threaded processes. Code, configurations and per-run results are in the repository [1].

4.3 Experimental Protocol

Algorithm comparison. *Phase 1* solves a one-day problem ($H = 1$, no day linking) and *Phase 2* the five-day problem ($H = 5$). Each configuration is solved in 10 independent runs $r = 1, \ldots, 10$ with seed $2026 + r$, shared by every method of that run; all values are averages over the 10 runs. The HiGHS time limit is 30 s in Phase 1 and $150\,\text{s} = 5 \times 30\,\text{s}$ in Phase 2. The metaheuristics have the same iteration budget: at most 300 iterations or generations, stopping after 50 consecutive iterations without improvement. In Phase 2, HiGHS is run twice per run, cold and warm-started from the ALNS solution of the same seed. When HiGHS proves optimality, the distance solve (3)–(4) follows with the same time limit. Before any timing, every method is run once on a five-customer instance so that Julia's compilation time is excluded.

Timing. The reported time is the wall-clock time of the solver call: `optimize!` for the main HiGHS solve, and construction, search and repair for the metaheuristics. Model building, the ALNS run that provides the warm start and the distance solve are excluded; the distance solve time is recorded separately [1].

Budget sensitivity. To check that the 300/50 budget does not decide the ranking, the Phase 2 runs on INS_S with $n = 80$ were repeated with five times the budget (1,500 iterations, 250 without improvement).

Policy comparison. The routing method is fixed per experiment: the best heuristic (ALNS) and the fastest one (Tabu) of Phase 2 at $n = 80$, both with the same 300/50 rule for the static plan and for every daily re-solve. The instance is INS_S ($n = 80$, $K = 2$, $H = 5$, $m_i = 2$) under three probability scenarios with 20 payment scenarios each. The dataset stores a uniform number $\xi_{it}^{(s)} \in [0, 1]$ per customer and day; a visit succeeds if $\xi_{it}^{(s)} < p_{it}$. All policies use the same numbers (common random numbers), and the planners never see them. In the static policy, a planned visit to a customer who has already paid is skipped and the freed time is not reused.

4.4 Evaluation Metrics

- **Expected collection** Z, **simulated collection** (sum of w_i over the successful visits in one payment scenario, used to compare policies), both in million VND, and **distance** in km.
- **Optimality gap.** With UB^* the tightest upper bound of the HiGHS calls in a run, the gap of every method in that run is

$$\text{gap} = \frac{UB^* - Z}{Z} \times 100\%. \tag{14}$$

  For the heuristics the gap is conservative, since UB^* may be loose. In all 200 runs every method returned a feasible solution and HiGHS a finite bound.
- **B&B nodes** (`MOI.NodeCount`) of the main solve, and the number of runs out of 10 with status `OPTIMAL`.

4.5 Algorithm Comparison

Table 2 reports the main instances; $n = 10$ and $n = 20$ behave like $n = 30$ (HiGHS optimal in 10/10 runs, every metaheuristic at the optimum), and all per-run data are in the repository [1].

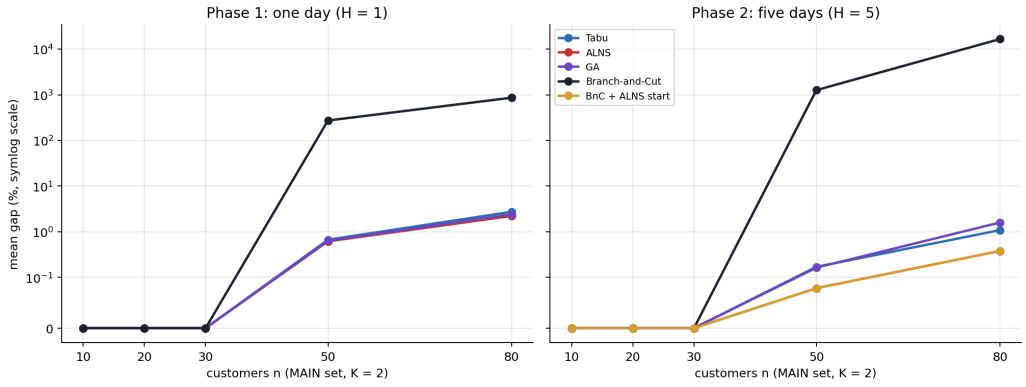


Figure 2: Mean gap (symmetric log scale) against n on the MAIN set.

Figure 2 plots the gap against n. Table 3 repeats the $n = 80$ comparison on independently generated customer sets, and Table 4 gives the budget check.

4.6 Static and Rolling-Horizon Comparison

Table 5 gives the paired differences with 95% confidence intervals (t distribution, 19 degrees of freedom). The expected value of the static ALNS plan was 287.94, 549.49 and 783.45 million VND in the low, base and high scenarios.

Table 2: Algorithm comparison over ten runs, mean ± standard deviation. Z in million VND, time in seconds, distance of the returned plan in km; nodes are means. "Warm" is HiGHS started from ALNS.

Instance	Method	Z	Time	Gap (%)	Nodes	Opt.	km
Phase 1: one day, 30 s limit							
INS_XS, $n = 30$	HiGHS	139.18 ± 0.00	1.96 ± 0.31	0.00 ± 0.00	1.0	10/10	62.6 ± 8.6
	GA	139.18 ± 0.00	3.00 ± 0.69	0.00 ± 0.00	–	–	53.3 ± 3.0
	Tabu	139.18 ± 0.00	1.37 ± 0.43	0.00 ± 0.00	–	–	53.0 ± 2.1
	ALNS	139.18 ± 0.00	1.32 ± 0.52	0.00 ± 0.00	–	–	51.2 ± 0.8
INS_S, $n = 50$	HiGHS	68.39 ± 41.62	30.80 ± 0.20	273 ± 171	3.8	0/10	33.6 ± 27.6
	GA	200.89 ± 0.24	6.77 ± 1.75	0.64 ± 0.12	–	–	64.8 ± 3.2
	Tabu	200.83 ± 0.28	2.44 ± 1.00	0.67 ± 0.15	–	–	66.2 ± 4.2
	ALNS	200.94 ± 0.23	7.03 ± 1.83	0.62 ± 0.14	–	–	64.2 ± 4.9
INS_S, $n = 80$	HiGHS	41.79 ± 30.79	31.60 ± 0.26	862 ± 742	0.0	0/10	21.1 ± 19.3
	GA	274.32 ± 0.96	7.97 ± 2.66	2.40 ± 0.37	–	–	65.5 ± 3.1
	Tabu	273.43 ± 0.52	2.50 ± 0.67	2.74 ± 0.24	–	–	68.9 ± 1.7
	ALNS	274.81 ± 0.61	14.32 ± 4.54	2.22 ± 0.22	–	–	63.6 ± 4.5
Phase 2: five days, 150 s limit							
INS_XS, $n = 30$	HiGHS	246.80 ± 0.00	16.85 ± 17.59	0.00 ± 0.00	56.5	10/10	144.0 ± 10.5
	HiGHS warm	246.80 ± 0.00	1.28 ± 0.18	0.00 ± 0.00	0.0	10/10	103.2 ± 2.3
	GA	246.80 ± 0.00	9.85 ± 2.03	0.00 ± 0.00	–	–	107.1 ± 3.3
	Tabu	246.80 ± 0.00	2.72 ± 1.31	0.00 ± 0.00	–	–	111.5 ± 3.2
	ALNS	246.80 ± 0.00	3.14 ± 0.69	0.00 ± 0.00	–	–	103.2 ± 2.3
INS_S, $n = 50$	HiGHS	35.82 ± 16.61	152.78 ± 0.41	$1{,}264 \pm 978$	4.1	0/10	55.5 ± 24.4
	HiGHS warm	361.90 ± 0.03	152.83 ± 0.29	0.08 ± 0.01	3.8	0/10	216.6 ± 6.6
	GA	361.59 ± 0.14	14.31 ± 3.77	0.16 ± 0.04	–	–	266.6 ± 9.4
	Tabu	361.56 ± 0.16	4.30 ± 1.72	0.17 ± 0.04	–	–	238.7 ± 6.5
	ALNS	361.90 ± 0.03	8.03 ± 2.97	0.08 ± 0.01	–	–	216.6 ± 6.6
INS_S, $n = 80$	HiGHS	6.99 ± 6.70	156.64 ± 0.27	$16{,}594 \pm 15{,}341$	0.0	0/10	4.2 ± 2.7
	HiGHS warm	549.04 ± 0.22	156.53 ± 0.28	0.38 ± 0.04	0.0	0/10	329.1 ± 12.4
	GA	542.49 ± 2.13	24.12 ± 4.45	1.59 ± 0.40	–	–	336.0 ± 9.6
	Tabu	545.20 ± 0.58	6.27 ± 1.05	1.09 ± 0.11	–	–	352.1 ± 6.7
	ALNS	549.04 ± 0.22	31.79 ± 6.55	0.38 ± 0.04	–	–	329.1 ± 12.4

Table 3: Gap (%) at $n = 80$ on independent customer sets, mean ± standard deviation over ten runs (HiGHS: mean).

Ph.	Instance	HiGHS	Warm	GA	Tabu	ALNS
1	INS_S	862	—	2.40 ± 0.37	2.74 ± 0.24	2.22 ± 0.22
	REP1_S	2,519	—	3.28 ± 0.50	3.63 ± 0.55	3.68 ± 0.77
	REP2_S	1,112	—	3.31 ± 0.32	3.71 ± 0.41	3.70 ± 0.47
	REP3_S	24,331	—	2.27 ± 0.38	2.57 ± 0.26	2.23 ± 0.34
	REP4_S	5,911	—	2.70 ± 0.10	2.73 ± 0.37	2.70 ± 0.12
2	INS_S	16,594	0.38 ± 0.04	1.59 ± 0.40	1.09 ± 0.11	0.38 ± 0.04
	REP1_S	41,049	0.49 ± 0.03	2.56 ± 0.30	1.18 ± 0.06	0.49 ± 0.03
	REP2_S	14,769	0.45 ± 0.03	2.21 ± 0.45	1.01 ± 0.06	0.45 ± 0.03

Table 4: Budget check, Phase 2, INS_S, $n = 80$: 300/50 against 1,500/250 iterations, mean ± standard deviation over ten runs.

	Gap (%)		Time (s)	
Method	1×	5×	1×	5×
GA	1.59 ± 0.40	0.87 ± 0.20	24 ± 4	181 ± 39
Tabu	1.09 ± 0.11	1.00 ± 0.09	6 ± 1	32 ± 6
ALNS	0.38 ± 0.04	0.33 ± 0.03	32 ± 7	238 ± 67

Table 5: Policy comparison on INS_S over 20 common payment scenarios. Static: mean simulated collection (million VND). Rolling and myopic: mean paired difference against static [95% CI], number of scenarios out of 20 in which the policy collected more, and change in distance.

			Rolling horizon		Myopic		km
Scen.	Method	Static	Diff [95% CI]	Better	Diff [95% CI]	Better	R / M
Low	ALNS	283.51	+4.9 [−3.0, 12.8]	14	+5.7 [−0.6, 11.9]	12	−5% / −12%
	Tabu	274.71	+9.7 [−0.3, 19.8]	10	+14.3 [−2.3, 31.0]	11	−6% / −17%
Base	ALNS	514.38	+2.3 [−4.3, 8.8]	12	−6.4 [−21.1, 8.3]	9	−9% / −17%
	Tabu	490.49	+4.4 [−13.2, 22.0]	11	**+19.9** [3.7, 36.2]	16	−6% / −17%
High	ALNS	735.90	+4.9 [−8.1, 18.0]	11	−2.5 [−23.4, 18.4]	9	−8% / −16%
	Tabu	687.06	+12.5 [−13.4, 38.4]	11	**+47.7** [20.9, 74.4]	16	−7% / −19%

Per payment scenario, the optimization time was 43–57 s for the static ALNS plan, 58–89 s for the five rolling-horizon re-solves and 28–54 s for the myopic ones; with Tabu the three figures were 5–9 s, 13–19 s and 5–9 s.

4.7 Discussion

Small instances. Up to 30 customers, HiGHS proves optimality in 10/10 runs in both phases and the three metaheuristics find the same value in every run. The one-day problem is solved at the root node, whereas the five-day problem with 30 customers needs 56.5 nodes and 16.9 s on average: linking the days makes the problem much harder. Started from the ALNS optimum, HiGHS needs only 1.3 s.

Distance. The distance solve did not help HiGHS. At $n = 20$ and $n = 30$ it always reached its time limit, and the cold HiGHS plan at $n = 30$ is longer than the ALNS plan with the same Z (62.6 against 51.2 km in Phase 1, 144.0 against 103.2 km in Phase 2). For $n \geq 50$ the cold HiGHS plan is short only because it visits few customers.

Behavior as size grows. The model grows roughly as $|K|\,H\,n^2$, from 2,012 variables ($n = 30$, one day) to 67,610 ($n = 80$, five days). From $n = 50$ on, HiGHS hits its time limit after at most 4 nodes on average, and its incumbent is poor (gaps of 273% to 16,594% on INS_S) and varies a lot between runs. The bound is still informative: the heuristics lie within 0.1–3.7% of it, so the large gaps reflect the weak incumbent, not a weak bound.

Quality versus runtime. At $n = 80$ the metaheuristics reach gaps of 2.2–3.7% in Phase 1 and 0.4–2.6% in Phase 2 in 2 to 42 s. In Phase 2, ALNS gives the smallest gap on every instance with $n \geq 50$ (0.08–0.49%), the smallest variation between runs and the shortest routes; Tabu is the fastest (4–7 s) at about 1%; GA is slower than Tabu and has the largest gap and variation at $n = 80$. In Phase 1 the three methods are within about 0.5 percentage points and none is best everywhere. The REP sets give the same picture. With five times the budget (Table 4) GA gains most (gap 1.59 to 0.87%) and passes Tabu, while ALNS and Tabu gain less than 0.1 point at 5–7.5 times the time. ALNS stays best, so the 300/50 budget does not change the main conclusion, but GA is the method most limited by it.

For $n \geq 50$, HiGHS started from ALNS returns exactly the ALNS solution: within 150 s it neither improves it nor closes the gap.

Benefits and limits of re-optimization. With ALNS, the rolling horizon collects 0.4–1.7% more than static in every scenario group, but no confidence interval excludes 0. Its clear benefit is travel: 5–9% less distance, because planned repeat visits to customers who have already paid are dropped. The myopic policy travels 12–17% less, but with ALNS its collection difference has a standard deviation of 13–45 million VND against 14–28 for the rolling horizon, so it sometimes wins and sometimes loses by a large margin. With Tabu, the static plan is weaker (3–7% below ALNS) and re-planning recovers much of the loss: the myopic policy collects significantly more than static in the base and high scenarios (+19.9 and +47.7 million VND, 16/20 scenarios), reaching the level of the ALNS policies. The one-day problems solved by the myopic policy are small, where Tabu is close to the optimum, while its five-day static plan is not. Re-planning thus matters most when the plan it replaces is weak. Two factors limit the gain: each re-solve optimizes an expectation as if no further re-planning will happen, and the variation between heuristic solves is of the same order as the gain. Finally, the collection varies far more between payment scenarios (standard deviation 150–220 million VND) than between policies, so only paired differences on common random numbers can separate them, and with 20 scenarios only differences of about 20 million VND are detectable.

5 Conclusion

This project modelled multi-officer, multi-day debt collection routing on the road network of Hanoi as a MILP in which one precomputed visit schedule is selected per customer, solved it with HiGHS, GA, Tabu search and ALNS, and compared a static plan with rolling-horizon and myopic re-planning.

Up to 30 customers, HiGHS proves optimality within seconds and should be preferred, since it gives a certificate. From 50 customers on, it stays at the root node within 30–150 s and its incumbent is unusable, but its bound still measures the heuristics. At this size ALNS gives the best five-day plans (gap 0.08–0.49%, also the shortest routes) and Tabu slightly weaker plans several times faster; five times more iterations do not change this. Warm-starting HiGHS from ALNS did not improve the ALNS plan. On 80 customers, re-planning on top of ALNS did not change the simulated collection significantly but cut travel by 5–9% (rolling horizon) and 12–17% (myopic); on top of the weaker Tabu plans, myopic re-planning raised the collection significantly in two of the three scenarios.

Three lessons follow. First, a fast heuristic re-run every morning can replace a strong one run once: Tabu with myopic re-planning collected within 2% of the static ALNS plan in every scenario, with 8–12% less travel and 5–10 times less computation. Second, the difficulty comes from linking the days rather than from the number of customers, since with 30 customers the one-day problem is solved at the root node while the five-day one needs 56 nodes; a decomposition by day is therefore the natural way to scale the exact model. Third, HiGHS fails at finding good integer solutions, not at bounding: its bound shows the heuristics to be at most 0.1–3.7% from the optimum, so the practical combination is a heuristic for the plan and the MILP bound as its quality certificate.

The customer accounts are synthetic, the payment probabilities are not calibrated on real outcomes, travel speeds ignore congestion and payments are full or nothing; the experiments stop at 80 customers, two officers and 20 payment scenarios. Natural next steps are the policy comparison with the 100 scenarios of the dataset on INS_M and INS_L, and stronger subtour-elimination cuts or a decomposition by day to keep exact methods usable beyond 30 customers.

References

[1] N. T. Phuc and D. T. L. Huong, *Mathematic Modelling Project: Source Code and Data*. Available at: `https://github.com/PhucNT2511/Mathematic-modeling-project`.

[2] OpenStreetMap contributors, *OpenStreetMap data for Vietnam* (Geofabrik extract `vietnam-latest.osm.pbf`, data up to 18 October 2026), Open Database License 1.0. Available at: `https://download.geofabrik.de/asia/vietnam.html`.

[3] NVIDIA, *Nemotron-Personas-Vietnam* [Dataset], CC BY 4.0, revision `950724d`, accessed 20 October 2026. Available at: `https://huggingface.co/datasets/nvidia/Nemotron-Personas-Vietnam`.

[4] I.-C. Yeh, *Default of Credit Card Clients* [Dataset], UCI Machine Learning Repository, 2009. DOI: `https://doi.org/10.24432/C55S3H`.

[5] C. E. Miller, A. W. Tucker, and R. A. Zemlin, "Integer programming formulation of traveling salesman problems," *Journal of the ACM*, vol. 7, no. 4, pp. 326–329, 1960.

[6] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.

[7] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization," *Mathematical Programming Computation*, vol. 15, pp. 581–589, 2023.

[8] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.